1 Evaluation Appendix

This appendix is a lookup-shaped reference for the evaluation chapter. The chapter references individual subsections by name as it needs them; each subsection below is self-contained and may be read in isolation.

The case study's source code, specifications, and grading infrastructure are split across three repositories. The task repository[1] carries the task specification, the participant package, and the containerised testing environment that participants download and run during their sessions. The grading-infrastructure repository[2] carries the automated grading suite, the transcript parser, and the submission-processing pipeline; it runs on the researcher's machine against each submission. The dataset repository[3] carries every submitted participant artefact in sanitised form: code patches, transcripts, and session metadata.

1.1 Reference Environments

The task is anchored on two open source codebases fixed to specific upstream commits and one containerised replica. Each commit and the replica serves a distinct role in the participant's session.

- `eclass-mcp-server` at commit `dbd2d16` is the patch baseline. The participant clones this repository at the start of each session, checks out the pin as a detached `HEAD`, and implements the task directly in the resulting working tree. The participant's submission patch is the diff between this baseline and the participant's staged changes; the grader applies that diff to the same pin and runs the test suite against the patched tree.
- `openeclass` at commit `e8b3329` is a read-only inspection reference. The participant clones this repository into the workspace so the assistant can inspect the legacy server's source during the task; no changes are made to it, and it is not part of any submission. The Docker replica (next item) carries the same openeclass codebase as this pin.
- The mock openeclass replica is a Docker image built from the upstream `Release_4.3.3` tag, whose codebase matches `e8b3329`, seeded with deterministic fixture data for courses, announcements, and users, and paired with a mock CAS service that replicates the Apereo CAS and SAML 1.1 surface openeclass's single sign-on integration expects. The replica is the integration target the participant's MCP server runs against during development and is the same replica the grader replays when scoring the submission.

A submission with no patch applied receives a baseline score of 9/100. Nine of the ten Python baseline tests (`P.1.1` through `P.1.9`) pass against the upstream reference; the tenth verifies the patch's tool registrations and requires the participant's diff to pass.

[1] Repository available at: https://github.com/sdi2200262/bsc-apm-case-study-task
[2] Repository available at: https://github.com/sdi2200262/bsc-apm-case-study-infra
[3] Repository available at: https://github.com/sdi2200262/bsc-apm-case-study-data

1.2 Submission Workflow

Each participant completes the task twice, each session time-boxed to three hours. For each session, the participant works against a clean clone of `eclass-mcp-server` pinned to the patch baseline commit (`dbd2d16`) as a detached `HEAD`, implements the task using the assigned framework, then generates the solution patch as the diff between the baseline and the participant's staged changes.

Between the two sessions the participant resets the working trees of both clones to their pins, clears untracked state, rebuilds the Python virtual environment, and wipes the local Claude Code project directory so the second session starts without any carried-over conversation state. Full step-by-step instructions for the setup, in-session, and reset procedures are provided in the participant guide that ships with the task repository.

1.2.1 Submission Format

Each submission is a single zip whose filename follows the pattern `{PID}_{S1|S2}_{apm|speckit}.zip`, where `PID` matches `P\d{3}` (for example, `P001`), the session field is `S1` or `S2`, and the framework field is `apm` or `speckit`. The zip places `solution.patch` and a `transcripts/` directory at its root with no wrapping directory.

Inside `transcripts/`, each main session transcript is a JSONL file accompanied by a sibling directory carrying the auxiliary records Claude Code persists for dispatched subagents and offloaded tool results; the parser consumes both.

A post-session questionnaire is filled per session alongside the zip upload through a single combined web form.

1.3 Post-Session Questionnaire Instrument

The post-session questionnaire is administered per session through the combined web form described in the Submission Format subsection above. All scored items use a five-point Likert scale.

The instrument captures the following items in both sessions:

- Prior familiarity with the framework before the session.
- Confidence in the correctness of the submitted solution.
- Perception of the framework's effectiveness on the task.
- Five workload items adapted from the NASA Task Load Index[4]: mental demand, effort, frustration, verification burden, and prompt-engineering burden.

The second session additionally elicits a perceived-difficulty comparison item: whether completing the task with the second framework felt easier, harder, or the same compared to the

[4]Instrument documentation available at: https://humansystems.arc.nasa.gov/groups/tlx/

first.

1.4 Data Privacy and Sanitisation

Each submission's transcripts are sanitised before they are appended to the dataset: a keyed symmetric encryption scheme is applied to the JSONL transcripts at rest, while the participant's `solution.patch` is preserved in plaintext alongside them. A small set of plaintext metadata (session identifier, canonical model identifier, record count, sanitisation timestamp, Claude Code version, and original payload size in bytes) is preserved alongside the encrypted payload, enabling dataset organisation and integrity validation without decryption. The exact cipher construction and metadata schema are documented in the parser repository's sanitisation module.

1.5 Per-Token Rate Card

The economic dimension of the evaluation values each session in dollars, computed by pricing every call recorded in the session's transcripts against Anthropic's per-token rate card. This subsection documents the rate-card snapshot the parser uses and the per-call attribution rules. The snapshot is dated **2026-04-25** and carries five rate categories per model: input tokens, output tokens, cache-read tokens, five-minute cache-write tokens, and one-hour cache-write tokens. The three models the cohort's sessions run on (Claude Opus 4.7, Claude Sonnet 4.6, and Claude Haiku 4.5) each carry the full five-category rate. The exact per-token prices, the rate-card sources, and the snapshot date are pinned in a YAML file the parser reads at every run, ensuring deterministic re-pricing of any historical session.

Cost is attributed per call. For each assistant call recorded in a transcript, the parser deduplicates by request identifier, reads the call's token usage across the five categories, and prices the call at the rate of the canonical model recorded on the call's record.

For calls made by subagents dispatched through the `Task` tool, the parser's primary resolution path opens the sibling `agent-<id>.jsonl` file the platform persists for the dispatch and prices every call in that file at the canonical model recorded inside it. The sibling files are what make per-call cost attribution deterministic for delegated dispatches; the dispatch-to-model index is built once per submission before pricing begins, so every dispatch whose sibling file is present resolves to a single model.

A fallback applies only when a sibling file is missing from the submission. The fallback resolves the model through three deterministic conventions in turn (an explicit declared model on the dispatch, a platform built-in default for the subagent type, or inheritance from the parent conversation); when none of the three applies, the dispatch's cost is reported as a closed interval bracketed by the cheapest and most expensive models on the platform, and the session cost is reported as lower and upper bounds rather than a point estimate.

1.6 APM Role Detection

APM operates as three agent roles (Planner, Manager, Worker), each in its own conversation; the case study reports per-role decompositions of the transcript-derived metrics so that an APM session's totals can be attributed to the role responsible for the work. This subsection documents how each conversation in an APM submission is assigned to a role.

Role assignment is derived from the first slash command issued in the conversation:

- `/apm-1-initiate-planner` marks the conversation as Planner.
- `/apm-2-initiate-manager` marks the conversation as Manager.
- `/apm-3-initiate-worker` marks the conversation as Worker.

Conversations whose first slash command is not one of these three role initiators are marked unassigned; they contribute to session totals but do not appear in the per-role decomposition.

Multiple conversations may carry the same role within a single submission: a Manager that handed off contributes successor Manager conversations, and parallel Workers each contribute their own conversation. Same-role conversations are numbered sequentially by their earliest timestamp (Manager-1, Manager-2; Worker-1, Worker-2), so the per-role view preserves the ordering observed in the session.

1.7 Framework Phase Detection

Each session of the case study is split into a planning portion and an implementation portion, and every transcript-derived metric is reported separately at each. This subsection documents how the planning-versus-implementation boundary is located in each framework's session. Because the two frameworks express phases differently, detection uses framework-specific rules applied at parse time.

For **APM**, the phase boundary follows from the role assignment described in the APM Role Detection subsection above: the Planner conversation is wholly assigned to the planning phase, and Manager and Worker conversations are wholly assigned to the implementation phase. Unassigned conversations contribute to the session totals but are excluded from the per-phase decomposition.

For **Spec-kit**, the phase assignment is per-prompt and follows a forward-only ratchet over an ordered triple of phases:

1. `other`: the initial phase before any phase-bearing command has been issued.
2. `planning`: entered when a planning-phase command is issued. The planning-phase commands are `/speckit-constitution`, `/speckit-specify`, `/speckit-plan`, `/speckit-tasks`, `/speckit-clarify`, `/speckit-analyze`, and `/speckit-checklist`.
3. `implementation`: entered when `/speckit-implement` is issued, the sole implementation-phase command.

A prompt carrying a command takes that command's phase; a prompt that does not carry a known command inherits the ratchet's current position.

Detection is applied independently per participant submission, and the per-phase metric values are aggregated at the cohort level in the results analysis.

1.8 Test Suite Composition

The test suite awards 100 points across two implementations, organised into hierarchical IDs of the form `P.x.y` (Python) and `C.x.y` (C replica).

The Python implementation carries 41 points across 24 tests in three sections:

- Baseline (10 points): 10 tests, P.1.1 through P.1.10, each worth one point, exercising protocol compliance and authentication against the upstream Python reference.
- Course-announcements feature (17 points): 8 tests, P.2.1 through P.2.8, covering input validation, output formatting, and aggregation behaviour.
- General-announcements feature (14 points): 6 tests, P.3.1 through P.3.6, covering input validation, output formatting, and per-entry source attribution.

The C replica carries 59 points across 30 tests in five sections:

- Build and technical stack (5 points): 2 tests, C.1.1 and C.1.2, verifying compilation against the required toolchain and the absence of forbidden dependencies.
- Baseline (10 points): 10 tests, C.2.1 through C.2.10, mirroring the Python baseline against the C replica.
- Course-announcements feature (17 points): 8 tests, C.3.1 through C.3.8, mirroring the Python course-announcements feature against the C replica.
- General-announcements feature (14 points): 6 tests, C.4.1 through C.4.6, mirroring the Python general-announcements feature against the C replica.
- Memory safety (13 points): 4 tests, C.5.1 through C.5.4, exercising the C replica under Valgrind.

Each test produces a pass-or-fail verdict against one of three assertion shapes. A strict assertion applies to response paths that return a single fixed string and checks the whole response text for exact equality with that constant. An implementation assertion applies to response paths that return a populated list of in-scope rows and checks that the response carries the seed-derived count preamble and the verbatim body of every in-scope row. An anchor assertion extends an implementation assertion along a feature-specific axis: for the general-announcements feature, it additionally requires the per-source attribution distribution across in-scope rows to match the expected distribution, and for the C replica's memory-safety section, it exercises the broadest multi-scenario Valgrind probe.

Within the course-announcements feature the row labelled as an anchor runs the same operational check as the implementation rows in that feature; the label captures the higher point weight of the row rather than an additional structural check.

Per-test point allocations, per-test definitions, and the source-level assertion logic are documented inside the grading-infrastructure repository.